

Computer Hardware

Deadlock and Cooperating Threads

Ksawery Moźdzynski

Agenda For Today



1. Deadlock

Why the bank stopped, and five hungry philosophers



2. Fixing it

A fixed lock order, `std::scoped_lock`



3. Waiting for a condition

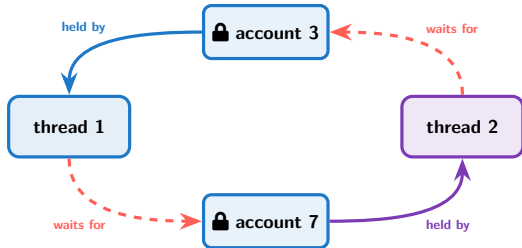
Condition variables and semaphores



4. Cooperating threads

Producer and consumer, a thread pool

Where we stopped: the bank stops



Step 2: one mutex for the bank
Correct, but a queue: one transfer at a time.

⚠ Step 3: a mutex per account
Sometimes it **hangs**: the messages stop, and the CPU usage drops to 0%.

The question

How can a thread lock **two** accounts without this happening? Today: the answer.

Task 0: one more time

Task

Run your `bank.cpp` from step 3 three times, with 4 threads.

1. How many runs finish?
2. While it hangs: what does Task Manager, Activity Monitor or `top` show for the CPU usage?

Task 0: one more time

Task

Run your `bank.cpp` from step 3 three times, with 4 threads.

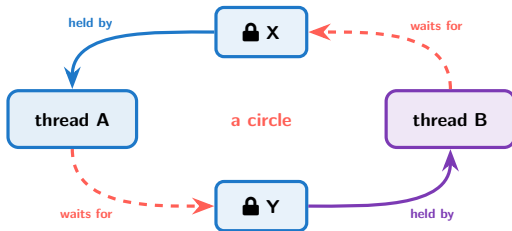
1. How many runs finish?
2. While it hangs: what does Task Manager, Activity Monitor or `top` show for the CPU usage?

✓ Solution

It never finishes. CPU usage: about 0%. The threads do not work: they are **Blocked**, they wait.

Deadlock

A **deadlock**: a group of threads where each one waits for something that **another one** in the group holds. Nobody can continue, ever.



The wait-for graph

An arrow from a lock to the thread that **holds** it, and from a thread to the lock it **waits for**.

A deadlock

In the graph, a deadlock is a **circle**.

Monday's scheduler cannot help: a Blocked task never gets a time slice.

Four conditions for a deadlock

1 Mutual exclusion

A lock has only **one** owner at a time.

2 Hold and wait

A thread **holds** one lock and **waits** for another.

3 No preemption

Nobody can take a lock **away** from a thread.

4 Circular wait

There is a **circle**: each thread waits for the next one.

A deadlock needs **all four**. Break **one** of them, and a deadlock is impossible.

The four conditions in the bank

	Condition	In the bank
1	Mutual exclusion	$m[3]$ has one owner
2	Hold and wait	thread 1 holds $m[3]$ and waits for $m[7]$
3	No preemption	nobody can take $m[3]$ from thread 1
4	Circular wait	thread 1 $\rightarrow m[7] \rightarrow$ thread 2 $\rightarrow m[3] \rightarrow$ thread 1

Conditions 1–3 are what a mutex **is**. The one we can break: **circular wait**.

Simulator 10: deadlock

Simulator task

Topic **Deadlock**: thread A locks X, then Y. Thread B locks Y, then X. You choose which thread runs the next step.

1. Find an order of steps that ends in a **deadlock**. How many steps does it take?
2. Find an order that finishes **without** a deadlock.
3. Switch thread B to *X then Y*. Can you still make a deadlock?

Solution: Simulator 10

Question 1

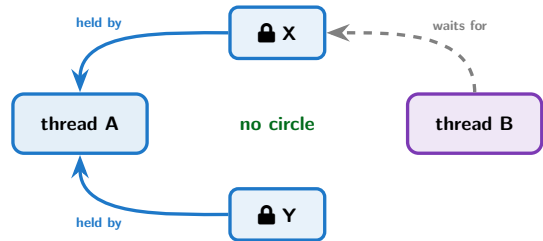
Two steps are enough: A locks X, B locks Y. Now A waits for Y and B waits for X: a circle.

Question 2

One thread takes **both** locks before the other one starts.

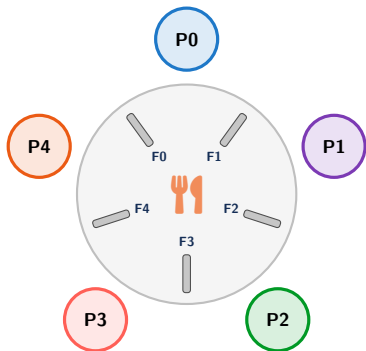
Question 3

No. Whoever gets X first also gets Y. The other one waits for X and holds **nothing**.



The same order of locks in every thread $\Rightarrow$ no circle $\Rightarrow$ no deadlock.

The dining philosophers



Five philosophers, five forks

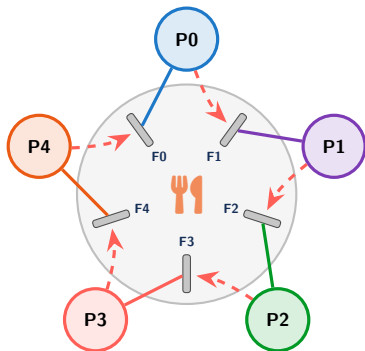
They think, get hungry and eat. To eat, a philosopher needs **both** forks next to him. A fork is a **mutex**.

```
● ● ●  
for (;;) {  
    think();  
    take the left fork;  
    take the right fork;  
    eat();  
    put both forks down;  
}
```

The question

Everybody gets hungry at the same moment. What happens?

Everybody takes the left fork



Each one holds a fork...

P0 holds F0, P1 holds F1, ..., P4 holds F4.

! ... and waits for the next one

P0 waits for F1, which P1 holds. P1 waits for F2, ..., and P4 waits for F0, which P0 holds.

— holds - - - waits for

Five philosophers, five forks, nobody eats. The same circle as in the bank, only bigger.

Task 1: the philosophers in C++

philosophers.cpp (a part)

```
● ● ●  
const int P = 5, MEALS = 1000;  
std::mutex fork_[P];  
int meals[P];  
  
void philosopher(int i) {  
    int left = i, right = (i + 1) % P;  
    for (int k = 0; k < MEALS; k++) {  
        std::lock_guard<std::mutex> a(fork_[left]);  
        // look around  
        std::lock_guard<std::mutex> b(fork_[right]);  
        meals[i]++; // eat  
    }  
}  
  
// main(): start 5 philosophers, join, print meals
```

Task

Download philosophers.cpp. Compile it with -O0.

1. Run it 5 times. Does it ever hang?
2. Replace the comment look around with a short pause:

```
std::this_thread::sleep_for(  
    std::chrono::microseconds(10));
```

Run it again.

Solution: the philosophers in C++

Question 1

On the test machine it **never** hung. The time between the two locks is so short that the circle almost never closes.

Question 2

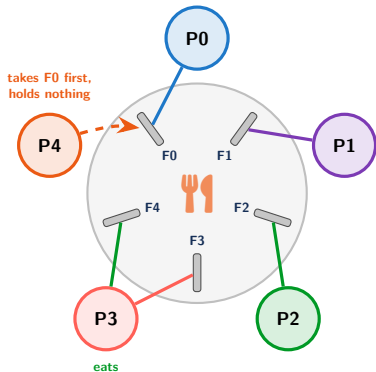
Now it hangs **every** time: all five hold the left fork while they look around.

⚠ A deadlock can hide

The program passes every test, and one day it hangs at a customer. **Testing cannot prove that there is no deadlock.**

We need a rule that makes the circle **impossible**, not just unlikely.

Rule: always lock in the same order



Give every lock a **number**. Every thread takes its locks in **increasing** order.

Why there is no circle

In a circle, the thread that holds the **highest** lock would wait for a **lower** one. The rule forbids that.



```
int first = std::min(left, right);  
int second = std::max(left, right);  
std::lock_guard<std::mutex> a(fork_[first]);  
std::lock_guard<std::mutex> b(fork_[second]);
```

Only P4 changes: it takes F0 first. One philosopher is enough to break the circle.

std::scoped_lock: two locks at once

```
● ● ●  
// in transfers(), around the transfer:  
std::scoped_lock both(m[from], m[to]);  
balance[from] -= x;  
balance[to]   += x;  
// both are unlocked at the end of the block
```

std::scoped_lock

Locks several mutexes **together**, without a deadlock, and unlocks them at the end of the block. (C++17)

How?

It takes one lock and **tries** the others. If one is busy, it gives back what it has and tries again.

Two tools: **your own order** (works everywhere, also on the ESP32), or `scoped_lock` (the simplest in C++).

Which condition do we break?

Method	Breaks
Lock order (by number)	④ circular wait
<code>std::scoped_lock</code>	② hold and wait: it never waits while it holds a lock
One big lock (Tuesday, step 2)	only one lock: a circle is impossible

Every working fix breaks one of the four conditions.

Task 2: fix the philosophers and the bank

Task

Fix both programs.

1. **Philosophers** (with the pause): take the forks in increasing order. Do they always finish? Does everybody eat 1000 times?
2. **Bank, step 4**: lock the two accounts in increasing order. Is the sum always 10 000?
3. **Bank, step 5**: use `std::scoped_lock` instead.
4. Measure steps 2, 4 and 5 with 1, 2 and 4 threads.

If you finish early: set `ACCOUNTS = 1000` and add slow work inside the lock. Compare step 2 and step 4 again, with 2 threads.

```
● ● ●  
volatile int sink;  
void slowWork() {  
    for (int k = 0; k < 500; k++)  
        sink = k;  
}
```

Solution: fix the philosophers and the bank

Step 4



```
int first = std::min(from, to);  
int second = std::max(from, to);  
std::lock_guard<std::mutex> a(m[first]);  
std::lock_guard<std::mutex> b(m[second]);
```

Questions 1–3

Always finishes, everybody eats 1000 times, the sum is always 10 000.

Question 4

With 10 accounts: no real gain over one big lock. Transfers often meet on the same account, and the work under the lock is tiny.

If you finished early

1000 accounts, longer work, 2 threads (a test machine): one big lock $\approx$ 1200 ms, a lock per account $\approx$ 470 ms.

Small locks pay off when threads **rarely** meet and the work under the lock is **long**. Always measure.

A new problem: wait until something is ready

So far: *do not disturb me* (a mutex). Now: *wake me up when it is ready*.



Busy waiting

```
while (!ready) { }
```



Sleep and check

```
while (!ready)  
sleep_for(100ms);
```

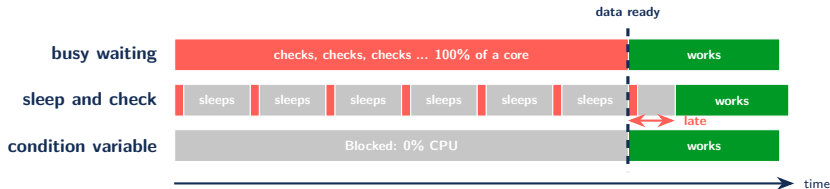


Wait until told

a condition variable

Remember polling and interrupts from Lectures 5 and 6? The same story again.

Busy waiting is polling



	CPU while waiting	Reacts
busy waiting	100% of a core	at once
sleep and check	almost 0%	up to 100 ms late
condition variable	0%	at once

The same story

Busy waiting = **polling**.

A condition variable = an **interrupt**: the other thread wakes you up.

std::condition_variable



```
std::mutex m;
std::condition_variable cv;
bool ready = false;

void worker() {
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock, [] { return ready; });
    // ... use the data ...
}

// in main(), when the data is ready:
{
    std::lock_guard<std::mutex> lock(m);
    ready = true;
}
cv.notify_one();    // wake up one thread
```

`cv.wait(lock, condition)`

If the condition is false: **unlock** the mutex and sleep (**Blocked**). When woken up: lock again and **check** again.

`notify_one()`, `notify_all()`

Wake up one, or all, waiting threads.

`std::unique_lock`

Like `lock_guard`, but `wait` can unlock it and lock it again.

Always give `wait` the condition: a thread may wake up *without* a `notify`, and a `notify` may come *before* the `wait`.

Task 3: three ways to wait

Task

Download `waiting.cpp`. `main` prepares the data for 3 seconds, then sets `ready`. The worker waits for it and prints how late it reacted. Version 1 is ready.

1. Run version 1. Look at the CPU usage while it waits.
2. Version 2: add `std::this_thread::sleep_for(std::chrono::milliseconds(100));` inside the loop. CPU usage? How late does the worker react?
3. Version 3: use a condition variable. CPU usage? How late does it react?

If you finish early: start 4 workers and wake them all up with one call. Which function do you need?

Solution: three ways to wait

Question 1

One core at **100%** for 3 seconds, just to check one variable.

Question 2

Almost 0%, but it reacts up to **100 ms** late.

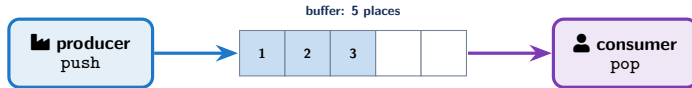
Question 3

0%, and it reacts **at once** (well below 1 ms).

If you finished early
`cv.notify_all()`.

Do not wait in a loop. Let another thread **wake you up**: wait and notify.

Producer and consumer



A **producer** puts data into a shared **buffer**, a **consumer** takes it out. They work at different speeds.

⚠ The buffer is full

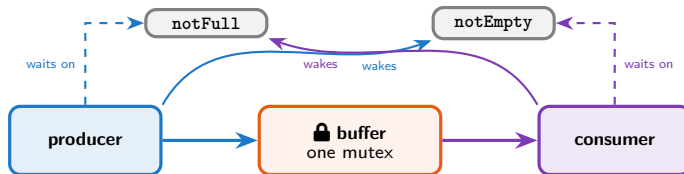
The producer must **wait** until a place is free.

⚠ The buffer is empty

The consumer must **wait** until there is data.

Examples: a keyboard and a program; a network card and the network stack; a camera and a video encoder; a button ISR and the LCD task.

The buffer: one mutex, two conditions



	waits for	after its work, wakes up
producer	<code>notFull</code>	<code>notEmpty</code>
consumer	<code>notEmpty</code>	<code>notFull</code>

Each one waits for its own condition, and wakes up the **other** one.

Simulator 11: producer and consumer

Simulator task

Topic **Producer-consumer**. Buffer size = 5.

1. The producer is **faster** than the consumer. What happens to the buffer? In which state is the producer most of the time?
2. The consumer is faster. And now?
3. Same speed, with *random short pauses* on. Change the buffer size from **1** to **8**. What does a bigger buffer give?

Solution: Simulator 11

Question 1

The buffer fills up. Then the producer is **Blocked** most of the time: it runs at the speed of the consumer.

Question 2

The buffer is almost always empty. The consumer is Blocked and waits for each item.

Question 3

A bigger buffer **absorbs** short pauses. With size 1 they wait for each other all the time.

In the long run the whole system runs at the speed of the **slower** one. The buffer only smooths out short differences.

The bounded buffer in C++



```
#include <queue>
#include <condition_variable>

const int SIZE = 5, ITEMS = 1000;
std::queue<int> buffer;
std::mutex m;
std::condition_variable notFull, notEmpty;

void producer() {
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notFull.wait(lock,
            [] { return buffer.size() < SIZE; });
        buffer.push(i);
        notEmpty.notify_one();
    }
}
```

std::queue

push adds at the back; front reads and pop removes at the front.

One mutex

The wait, the push and the notify: all under one mutex.

Task 4: the bounded buffer

Task

Download `buffer.cpp`. It has the producer from the slide before. Write `consumer()`.

1. The consumer takes `ITEMS` numbers from the buffer and adds them up. Expected sum:
 $1 + 2 + \dots + 1000 = \mathbf{500\,500}$. Is your sum correct?
2. In the producer, print a message when it has to wait (the buffer is full). How often does it wait? Try `SIZE = 1` and `SIZE = 100`.
3. What happens if you forget `notFull.notify_one()` in the consumer?

If you finish early: two producers (1 ... 1000 each) and two consumers. Expected sum: 1 001 000. How does each consumer know when to stop?

Solution: the bounded buffer



```
long long consumer() {
    long long sum = 0;
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notEmpty.wait(lock,
            [] { return !buffer.empty(); });
        sum += buffer.front();
        buffer.pop();
        notFull.notify_one();
    }
    return sum;
}
```

Questions 1 and 2

500 500. The smaller the buffer, the more often the producer waits.

Question 3

Once the buffer is full, the producer sleeps and **nobody wakes it up**: the program hangs. No circle of locks, but the same effect: a thread waits forever.

If you finished early

Each consumer takes 1000 items. Or: at the end, the producers put a special value (e.g. -1, a *poison pill*) for each consumer.

Semaphores



```
● ● ●  
#include <semaphore> // C++20  
  
std::counting_semaphore<3> parking(3);  
  
void car(int id) {  
    parking.acquire(); // wait for a place  
    // ... park ...  
    parking.release(); // leave  
}
```

A **semaphore** counts free places. `acquire()`: take a place, or wait if there is none. `release()`: give a place back and wake up one waiting thread.

A mutex and a semaphore

A mutex: **one** owner. A semaphore: up to **N** at the same time. On Tuesday you saw one in FreeRTOS: `xSemaphoreCreateMutex`.

The buffer with two semaphores

Producer



```
std::counting_semaphore<SIZE> freeSlots(SIZE);
std::counting_semaphore<SIZE> fullSlots(0);

// for each item:
freeSlots.acquire();           // a free place
{
    std::lock_guard<std::mutex> lock(m);
    buffer.push(i);
}
fullSlots.release();           // one more item
```

Consumer



```
// for each item:
fullSlots.acquire();           // an item
{
    std::lock_guard<std::mutex> lock(m);
    sum += buffer.front();
    buffer.pop();
}
freeSlots.release();           // one more place
```

The semaphores count the places and the items. The mutex protects the queue itself.

Task 5: a semaphore

Task

A print room with 2 printers.

1. Start **8** threads. Each one prints a document: `sleep_for(500ms)`. At most **2** threads may print at the same time: use a `std::counting_semaphore`.
2. Count in a `std::atomic<int>` inside how many threads are printing right now, and print it. Is it ever more than 2?
3. How long does the whole program take? **Calculate first**, then measure.

If you finish early: change Task 4 to the version with two semaphores. Is the sum still 500 500?

Solution: a semaphore



```
std::counting_semaphore<2> printers(2);
std::atomic<int> inside{0};

void print(int id) {
    printers.acquire();
    int now = ++inside;
    std::cout << id << " prints, inside: "
              << now << "\n";
    std::this_thread::sleep_for(
        std::chrono::milliseconds(500));
    --inside;
    printers.release();
}
```

Question 2

Never more than 2.

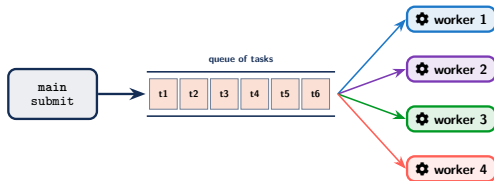
Question 3

8 documents, 2 at a time, 0.5 s each: 4 rounds, about 2 s.

If you finished early

Yes, 500 500.

A thread pool



A **thread pool**: a fixed group of threads (*workers*) that take tasks from a shared queue.

Why?

Starting a thread costs time. Thousands of small tasks do not need thousands of threads, only as many as there are **cores**.

What is it?

A producer and consumers: **main** produces tasks, the workers consume them.

On Monday the scheduler gave tasks to **one core**. A thread pool gives tasks to **threads**: your own small scheduler.

The pool in C++

```
class ThreadPool {
public:
    explicit ThreadPool(int n) {
        for (int i = 0; i < n; i++)
            workers.emplace_back([this] { work(); });
    }
    void submit(std::function<void()> task) {
        { std::lock_guard<std::mutex> lock(m);
          tasks.push(task); }
        cv.notify_one();
    }
    ~ThreadPool(); // TODO: stop and join the workers
private:
    void work(); // TODO: take a task, run it, repeat
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex m;
    std::condition_variable cv;
    bool stop = false;
};
```

`std::function<void()>`

Any function without arguments, for example a lambda.

`[this] { work(); }`

Each worker runs `work()` of this pool.

The destructor

`~ThreadPool()` runs when the pool goes out of scope.

Task 6: count primes with a pool

main() in pool.cpp

```
● ● ●  
const int LIMIT = 5'000'000, CHUNK = 100'000;  
std::atomic<int> primes{0};  
{  
    ThreadPool pool(WORKERS);  
    for (int start = 0; start < LIMIT;  
         start += CHUNK)  
        pool.submit([start, &primes] {  
            int c = 0;  
            for (int n = start;  
                 n < start + CHUNK; n++)  
                if (isPrime(n)) c++;  
            primes += c;  
        });  
} // the destructor waits for all tasks
```

Task

Download pool.cpp. Write work() and the destructor. Compile with -O2.

1. work(): wait for a task **or** for stop; take the task, **unlock**, run it.
2. The destructor: set stop, wake up **all** workers, join them.
3. Measure with 1, 2, 4 and hardware_concurrency() workers. Expected: **348 513** primes.

If you finish early: why must the task run **after** the unlock?

Solution: count primes with a pool

```
void work() {  
    while (true) {  
        std::function<void()> task;  
        {  
            std::unique_lock<std::mutex> lock(m);  
            cv.wait(lock, [this] {  
                return stop || !tasks.empty(); });  
            if (stop && tasks.empty()) return;  
            task = tasks.front();  
            tasks.pop();  
        }  
        task();  
    }  
}  
  
~ThreadPool() {  
    { std::lock_guard<std::mutex> lock(m);  
      stop = true; }  
    cv.notify_all();  
    for (auto &w : workers) w.join();  
}
```

Time (a test machine, 2 cores)

Workers	Time
1	2050 ms
2	960 ms
4	1020 ms

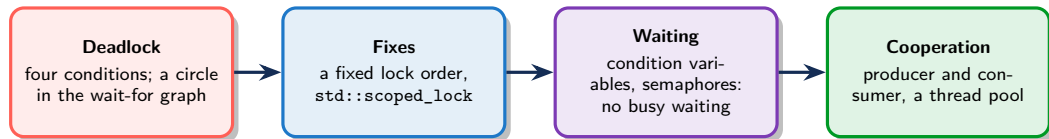
Question 3

Up to the number of cores: $2\times$ faster on 2 cores. More workers than cores: no gain.

If you finished early

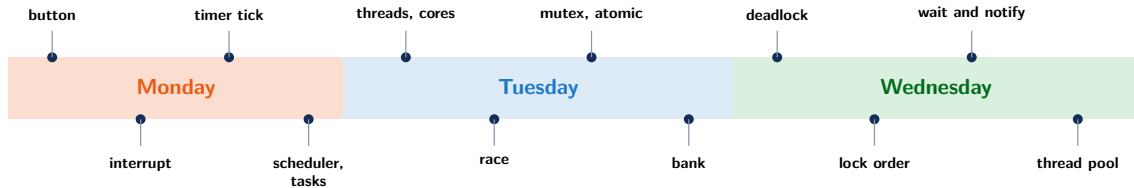
Under the lock, the other workers could not take tasks: they would run one after another again.

What we saw today



Threads must not only stay out of each other's way. They must also **work together**.

Three days: from a button to a thread pool



The same ideas, twice
polling ↔ busy waiting
an interrupt ↔ notify

a Blocked task in FreeRTOS ↔ a thread in wait
the scheduler ↔ a thread pool

A few ideas, from a small microcontroller to a laptop with many cores.

Mini-project (if there is time)

1 The auditor (C++)

Today's bank, plus one more thread that checks the sum **every 100 ms** while the transfers run. It must lock **all** accounts, without a deadlock.

Hint: the lock order.

2 A pipeline (C++)

Three threads connected by two bounded buffers:

generate numbers → square them → add them up.

Check the result with a formula.

3 Back to the ESP32

Button A's ISR puts each press into a FreeRTOS queue (xQueueSendFromISR). A task takes them out (xQueueReceive) and shows them on the LCD.

A producer and a consumer on the microcontroller.

Choose one. Projects 1 and 2 on your laptop, project 3 in Wokwi with your project **Circuit**.

Check yourself

1. What are the four conditions for a deadlock? Which one does a fixed lock order break?
2. Why did the philosophers' program hang only with the pause? What does it tell you about testing?
3. Why is busy waiting bad? What does `cv.wait(lock, condition)` do with the mutex?
4. When does the producer wait, and when does the consumer wait?
5. What is the difference between a mutex and a semaphore?
6. Why does a pool with more workers than cores not get faster?

Try to answer each question in **2–3 sentences**.